DATA ENGINEERING

Data Lineage Visual Guide

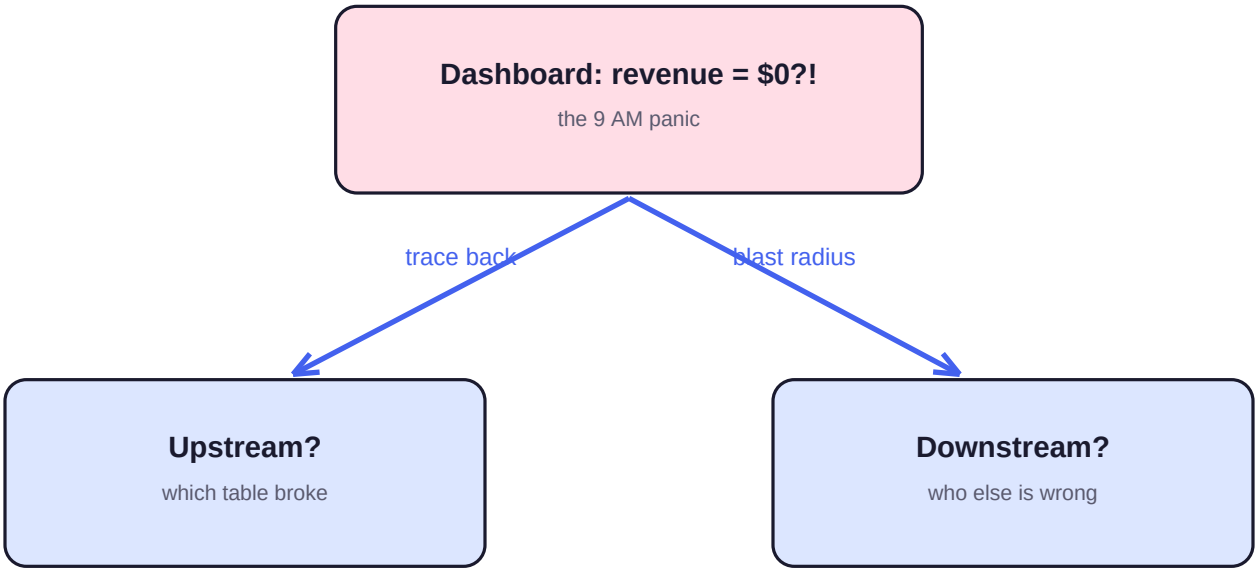
Where did this number come from?
Column-level tracking and impact analysis.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

The Question Lineage Answers

Every data incident starts with 'where did this come from?'



Without lineage: hours of Slack archaeology. With lineage: click the metric, see the full upstream chain and every downstream consumer.

STEP 2/6

Lineage Granularity Levels

Table-level tells you what; column-level tells you why.

Table-level

orders → revenue_mart (coarse, easy)

Column-level

orders.total → revenue_mart.gross_revenue (precise, hard)

Column-level lineage parses SQL to map each output column to its source columns through every transform — including through CASE statements, joins, and aggregations.

Column-level is what makes impact analysis trustworthy: 'we're renaming total' → exactly which dashboards break, not 'everything downstream of orders'.

STEP 3/6

How Lineage Is Captured

Parse it, instrument it, or declare it.

SQL parsing

Parse query text into column mappings. Works on any SQL, struggles with dynamic SQL.

Framework instrumentation

Spark/dbt emit lineage events (OpenLineage). Precise, needs integration.

Manual declaration

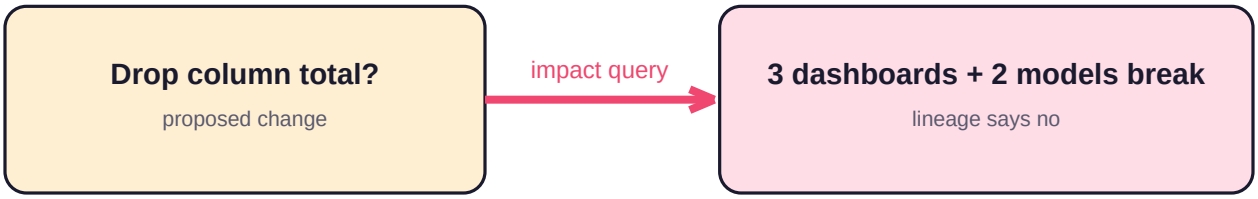
Humans document flows. Accurate once, rots immediately.

Best practice: OpenLineage events from orchestrators (Airflow, Spark, dbt) into a central catalog — automatic, versioned, and queryable.

STEP 4/6

Impact Analysis: Before You Change Anything

Lineage turns 'can I drop this column?' into a 10-second query.



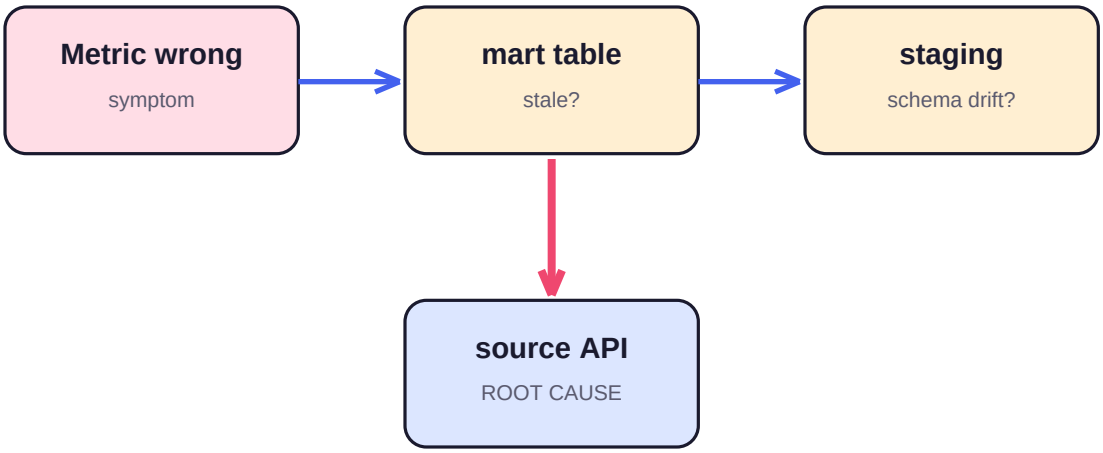
Forward lineage (downstream) for impact analysis; backward lineage (upstream) for root-cause. CI can block breaking changes automatically.

- Deprecation workflow: announce → migrate → verify zero consumers → drop
- Schema-change CI: fail the PR if a dropped column has downstream readers
- Blast-radius estimates for incident severity

STEP 5/6

Incident Debugging with Lineage

Walk the graph from symptom to cause.



The debugging walk: check freshness at each hop, find the first broken node. Lineage plus data-quality signals usually point at the culprit in minutes.

- Annotate nodes with freshness + test status — the graph becomes a health map
- Common culprits: missed SLA upstream, schema drift, silent filter change
- Post-incident: the fix should add a check at the hop that failed

STEP 6/6

Lineage × Governance

Lineage is how PII policies and audits actually work.

PII propagation

ssn flows into 14 tables

Compliance proof

GDPR deletion = all copies

Tag a source column as PII → lineage propagates the tag downstream automatically. Masking policies follow the tag, not the table name.

Right-to-erasure without lineage is a guess. With column-level lineage, 'delete this user's data everywhere' is a precise, auditable operation.

STEP ✓/✓

Key Takeaways

Data Lineage — Remember

- Lineage answers 'where did this come from' and 'who breaks if I change this' — the two incident questions.
- Column-level lineage maps each output column through every transform; table-level is a coarse start.
- Capture automatically: OpenLineage events from Spark/dbt/Airflow beat manual documentation.
- Forward lineage drives impact analysis and safe deprecation; backward lineage drives root-cause debugging.
- Lineage powers governance: PII tags propagate downstream, and erasure becomes auditable.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev